Refactoring

Python

```
class Order:
    def __init__(self, order_amount):
        self.order_amount = order_amount

    def calculate_discount(self, customer_type, is_first_time_buyer, location):
        order_amount = self.order_amount
        if customer_type == "VIP" and is_first_time_buyer and location == "Local":
            discount = order_amount * 0.20
        elif customer_type == "Regular" and location == "Local":
            discount = order_amount * 0.10
        else:
            discount = 0

        # Apply tax
        tax = order_amount * 0.05
        final_amount = order_amount - discount + tax
        return final_amount

class PaymentProcessor:
    def __init__(self, order):
        self.order = order

    def process_payment(self, payment_method):
        amount_due = self.order.calculate_discount()
        print(f"Processing payment of {amount_due} using {payment_method}.")
        print("Payment processed successfully.")
```

Java

```
class Order {
    public double order_amount;

    public Order(double order_amount) {
        this.order_amount = order_amount;
    }

    public double calculate_discount(String customer_type, boolean is_first_time_buyer, String location)
{
        double order_amount = this.order_amount;
        double discount;
        if (customer_type.equals("VIP") && is_first_time_buyer && location.equals("Local")) {
            discount = order_amount * 0.20;
        } else if (customer_type.equals("Regular") && location.equals("Local")) {
            discount = order_amount * 0.10;
        } else {
            discount = 0;
        }
```

```
        // Apply tax
        double tax = order_amount * 0.05;
        double final_amount = order_amount - discount + tax;
        return final_amount;
    }
}

class PaymentProcessor {
    private Order order;

    public PaymentProcessor(Order order) {
        this.order = order;
    }

    public void process_payment(String payment_method) {
        double amount_due = order.calculate_discount();
        System.out.println("Processing payment of " + amount_due + " using " + payment_method + ".");
        System.out.println("Payment processed successfully.");
    }
}
```

1. Identify the code smells in the above implementation. **(THE NEXT PART IS SHOWN HERE AS PRACTICE BUT NOT USUALLY PART OF THE QUESTION)** Explain what problem each of them indicate.

2. Suggest remedies to address the identified smells. **(THE NEXT PART IS SHOWN HERE AS PRACTICE BUT NOT USUALLY PART OF THE QUESTION)** Explain how these changes improve the code.

3. Refactor the code to resolve all identified issues. Write only the corrected code.

Answer:

Long Method

The calculate_discount method is lengthy and contains a complex conditional structure.

Problem: It is harder to understand and maintain, as the logic is buried within a single block of code. This increases the chance of errors when modifications are required.
Remedy: Extract Method and Decompose Conditional to simplify the logic and make each part self-explanatory.

Long Parameter List

The calculate_discount method takes multiple parameters (customer_type, is_first_time_buyer, location).

Problem: It makes the method hard to use and prone to errors due to the need to pass multiple values explicitly, which reduces code readability and usability.

Remedy: Introduce Parameter Object to group related parameters into a single object, improving method clarity.

Feature Envy
The process_payment method accesses multiple attributes of Order and external inputs without encapsulating the logic.

Problem: It creates tight coupling by scattering logic across classes, making it harder to maintain and test the code.
Remedy: Move Method to the class where the data resides, aligning behavior with the relevant data.

Inappropriate Naming
Class Order and method calculate_discount lack specificity about their purpose.

Problem: It reduces readability and makes it harder for developers to understand their roles at a glance.
Remedy: Rename for clarity to ensure names convey the intention and functionality explicitly.

Refactored Code:

```
class CustomerInfo:
    def __init__(self, customer_type, is_first_time_buyer, location):
        self.customer_type = customer_type
        self.is_first_time_buyer = is_first_time_buyer
        self.location = location

class Order:
    def __init__(self, amount, customer_info):
        self.amount = amount
        self.customer_info = customer_info

    def calculate_amount_after_discount(self):
        discount = self._determine_discount()
        tax = self._determine_tax()
        return self.amount - discount + tax

    def _determine_discount(self):
        if self._is_vip_local_first_time():
            return self.amount * 0.20
        elif self._is_regular_local():
            return self.amount * 0.10
        return 0

    def _determine_tax(self):
        return self.amount * 0.05

    def _is_vip_local_first_time(self):
```

```
    return (self.customer_info.customer_type == "VIP" and
            self.customer_info.is_first_time_buyer and
            self.customer_info.location == "Local")

def _is_regular_local(self):
    return (self.customer_info.customer_type == "Regular" and
            self.customer_info.location == "Local")

def process_payment(self, payment_method):
    amount_due = self.calculate_amount_after_discount()
    print(f"Processing payment of {amount_due} using {payment_method}.")
    print("Payment processed successfully.")
```

Python

```python
class ReportGenerator:
    def __init__(self, data):
        self.d = data

    def generate_summary(self):
        # This method generates a summary of the data
        total = 0
        for item in self.d:
            total += item
        average = total / len(self.d)
        print(f"Total: {total}")
        print(f"Average: {average}")

    def generate_detailed_report(self):
        # This method generates a detailed report of the data
        total = 0
        for item in self.d:
            total += item
            print(f"Data Point: {item}")
        average = total / len(self.d)
        print(f"Detailed Total: {total}")
        print(f"Detailed Average: {average}")
```

Java

```java
class ReportGenerator {
    private int[] d;

    public ReportGenerator(int[] data) {
        this.d = data;
    }

    public void generateSummary() {
        // This method generates a summary of the data
        int total = 0;
        for (int item : d) {
            total += item;
        }
        double average = (double) total / d.length;
        System.out.println("Total: " + total);
        System.out.println("Average: " + average);
    }

    public void generateDetailedReport() {
        // This method generates a detailed report of the data
        int total = 0;
        for (int item : d) {
            total += item;
            System.out.println("Data Point: " + item);
        }
```

```
        double average = (double) total / d.length;
        System.out.println("Detailed Total: " + total);
        System.out.println("Detailed Average: " + average);
    }
}
```

1. Identify the code smells in the above implementation. **(THE NEXT PART IS SHOWN HERE AS PRACTICE BUT NOT USUALLY PART OF THE QUESTION)** Explain what problem each of them indicate.
2. Suggest remedies to address the identified smells. **(THE NEXT PART IS SHOWN HERE AS PRACTICE BUT NOT USUALLY PART OF THE QUESTION)** Explain how these changes improve the code.
3. Refactor the code to resolve all identified issues. Write only the corrected code.

Answer:

Duplicated Code

Both generate_summary and generate_detailed_report calculate the total and average in the same way.

Problem: Redundant logic leads to increased maintenance effort and a higher chance of bugs.
Remedy: Extract Method to centralize the calculation of total and average.

Inappropriate Naming

The attribute d is not descriptive.

Problem: Reduces code readability and understanding.
Remedy: Rename to data to clearly indicate its purpose.

Long Method

The methods include multiple responsibilities (calculating and printing).

Problem: Reduces clarity and makes methods harder to maintain.
Remedy: Extract Method to separate concerns of calculation and printing.

Refactored Code:

```
class ReportGenerator:
    def __init__(self, data):
        self.data = data

    def generate_summary(self):
        total, average = self._calculate_summary()
        self._print_summary(total, average)
```

```
def generate_detailed_report(self):
    total, average = self._calculate_summary()
    self._print_detailed_report(total, average)

def _calculate_summary(self):
    total = sum(self.data)
    average = total / len(self.data)
    return total, average

def _print_summary(self, total, average):
    print(f"Total: {total}")
    print(f"Average: {average}")

def _print_detailed_report(self, total, average):
    for item in self.data:
        print(f"Data Point: {item}")
    print(f"Detailed Total: {total}")
    print(f"Detailed Average: {average}")
```